THE PREMIUM GUIDE

BACKEND INTERVIEW QUESTIONS.

10 topics. 80 questions. The exact questions asked in backend interviews at product companies — with what interviewers are actually testing for in each answer.

TOPICS COVERED

01	Database Indexing	●●●●●	**06**	Message Queues	●●●●■
02	Transactions & Isolation	●●●●●	**07**	Distributed Systems	●●●●■
03	Concurrency	●●●●■	**08**	API Design	●●●●●
04	Caching	●●●●●	**09**	Database Design	●●●●■
05	Idempotency	●●●●■	**10**	Production Debugging	●●●●●

HOW EACH PAGE IS STRUCTURED

Every topic gets one page: what the concept is, 8 real interview questions asked frequently, and — most importantly — what the interviewer is actually testing for in each answer. Not just Q&A. The reasoning behind it.

TOPIC 01 ●●●●●

ASKED EVERYWHERE

Database Indexing

Indexing is the #1 performance topic in backend interviews. Interviewers don't want definitions — they want to know if you've actually run EXPLAIN, seen a slow query, and fixed it.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What is a B-tree index and how does it work internally?

■ *They want to know if you understand it's a sorted data structure, not just 'makes queries faster'.*

02 What is the difference between a clustered and non-clustered index?

■ *Clustered = data rows stored in index order (one per table). Non-clustered = pointer to the row. Common confusion point.*

03 What is a composite index? Explain the left-prefix rule.

■ *An index on (A, B, C) only helps queries filtering on A, or A+B, or A+B+C. Not B alone. This trips people up constantly.*

04 When would you NOT add an index?

■ *High write volume tables, low cardinality columns (boolean, status with 3 values), small tables. They test if you know the write cost.*

05 What is a covering index?

■ *An index that contains all columns a query needs — so the DB never touches the actual table. Key for read-heavy APIs.*

06 How do you diagnose a slow query in production?

■ *EXPLAIN / EXPLAIN ANALYZE. Look for Seq Scan on large tables, high row estimates, missing index. Then verify with actual query time.*

07 What is index cardinality and why does it matter for query planning?

■ *High cardinality (unique values) = index is useful. Low cardinality (3 values) = full table scan may be faster. Query planner uses stats.*

08 How does an index affect INSERT, UPDATE, and DELETE performance?

■ *Every write must also update every index. 10 indexes = 10x write overhead. Interviewers test if you understand this trade-off.*

TOPIC 02 ●●●●●

ASKED EVERYWHERE

Transactions & Isolation Levels

This is one of the most misunderstood backend topics. Most developers use transactions but can't explain what isolation level their app runs at or what bugs that might cause.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What are ACID properties? Explain each one with a real example.

- *Atomicity: all or nothing. Consistency: valid state to valid state. Isolation: concurrent transactions don't see each other's work. Durability: committed = persisted.*

02 What are the 4 isolation levels? What problem does each one prevent?

- *READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE. Each prevents a class of anomaly. Most DBs default to READ COMMITTED.*

03 What is a dirty read, phantom read, and non-repeatable read?

- *Dirty: reading uncommitted data. Non-repeatable: same query returns different values in same transaction. Phantom: new rows appear between two reads.*

04 What is a deadlock? How do you detect and prevent one?

- *Two transactions each waiting for the other's lock. Detection: DB kills one. Prevention: consistent lock ordering, timeouts, keep transactions short.*

05 What is the difference between optimistic and pessimistic locking?

- *Pessimistic: lock the row before reading. Optimistic: read freely, check version on write, retry if changed. Optimistic scales better under low contention.*

06 When would you use SERIALIZABLE isolation and what does it cost you?

- *When you can't tolerate any anomalies (financial systems). Cost: serialisation failures, retries, dramatically lower throughput. Rarely the right default.*

07 What is a two-phase commit and what problem does it solve?

- *Coordinates a transaction across multiple databases: prepare phase (all agree), commit phase (all execute). Solves distributed atomicity. High latency cost.*

08 What is a savepoint? When would you use one?

- *A rollback point within a transaction. Roll back part of a transaction without aborting the whole thing. Useful for nested operations with partial failure handling.*

TOPIC 03 ●●●●○

VERY COMMON

Concurrency

Concurrency bugs are the hardest to reproduce and the most expensive in production. Interviewers use this topic to test if you understand what actually happens when two requests hit the same code at the same millisecond.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What is a race condition? Give a real backend example.

- *Two requests read a balance (100), both deduct 50, both write 50 back. Result: 50 not 0. Classic lost update. Shows up in payments, inventory, counters.*

02 What is the difference between concurrency and parallelism?

- *Concurrency: dealing with multiple things at once (interleaved, one CPU). Parallelism: doing multiple things at once (multiple CPUs). Node.js is concurrent, not parallel.*

03 How does Node.js handle concurrent requests without multiple threads?

- *Event loop + non-blocking I/O. One thread processes callbacks. I/O operations are handed to the OS and a callback fires when done. CPU-bound work blocks the loop.*

04 What is an atomic operation? Why does it matter for counters?

- *Executes as a single indivisible unit. Redis INCR is atomic. SQL UPDATE counter = counter + 1 is atomic. Reading then writing is not. Interviewers test this gap.*

05 What is a connection pool and what happens if you don't use one?

- *Reuses DB connections instead of opening a new one per request. Without it: connection overhead per request, DB runs out of connections under load. Always use one.*

06 What is a distributed lock and when do you need one?

- *A lock across multiple service instances. Use when multiple instances might process the same job or modify the same resource. Redis SETNX or Redlock algorithm.*

07 What is the thundering herd problem in the context of concurrency?

- *Many threads/processes all wake up at once (cache expiry, lock release) and slam a resource. Solution: jitter, probabilistic early expiry, single lock for cache refresh.*

08 How do you make a counter increment safe under high concurrency?

- *Database atomic update (UPDATE SET count = count + 1), Redis INCR, or a dedicated counter service. Never: read, increment in app, write back.*

TOPIC 04 ●●●●●

ASKED EVERYWHERE

Caching

Caching questions are in almost every senior backend interview. The wrong answers are 'use Redis' and 'add a TTL'. The right answers involve strategy, invalidation, and failure modes.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 Explain the four main caching strategies and when to use each.

- *Cache-aside (app manages), write-through (write to cache + DB together), write-behind (write to cache, async to DB), refresh-ahead (preload before expiry). Each has a failure mode.*

02 What is cache invalidation and why is it considered one of the hardest problems in CS?

- *Knowing when to remove or update a cached value. Invalidate too early = cache miss storm. Too late = stale data. No universal solution. Strategy depends on staleness tolerance.*

03 What is a cache stampede / thundering herd and how do you prevent it?

- *Many requests hit a cold cache simultaneously after expiry and all go to DB. Prevention: mutex lock on cache rebuild, probabilistic early expiry, staggered TTLs.*

04 What is a hot key problem in Redis and how do you solve it?

- *One key gets 90% of traffic and overloads a single Redis node. Solution: local in-memory cache for hot keys, key sharding (append shard ID), read replicas for Redis.*

05 When should you NOT cache something?

- *Data that changes faster than the TTL, financial balances that must be exact, user-specific data that varies per request, anything where staleness causes security issues.*

06 What is the difference between a local cache and a distributed cache?

- *Local: in-process, fast, inconsistent across instances. Distributed (Redis): shared, consistent, network overhead. Use local for session-local data, distributed for shared state.*

07 How do you handle cache consistency when data changes?

- *Event-driven invalidation (publish an event on write, consumers invalidate), write-through (update cache on every write), versioned keys (new key on change, old one expires).*

08 What is a CDN and how is it different from an application-level cache?

- *CDN caches at the edge, close to users, for static and cacheable HTTP responses. App cache (Redis) is for data. CDN reduces network latency; Redis reduces DB load.*

TOPIC 05 ●●●●○

VERY COMMON

Idempotency

Idempotency is a senior-level topic that most junior devs have never heard of and most mid-level devs know the word but can't implement. It comes up in every payments, messaging, and distributed systems interview.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What is idempotency? Why does it matter for APIs?

■ *Calling an operation N times has the same effect as calling it once. Matters because networks are unreliable: clients retry. Without idempotency, retries cause duplicate payments, duplicate emails, double charges.*

02 Which HTTP methods are idempotent by spec? Which aren't?

■ *GET, PUT, DELETE, HEAD are idempotent. POST is not. PATCH is not by spec (though it can be designed to be). GET = read only. PUT = replace, not append.*

03 How do you implement idempotency for a payment API endpoint?

■ *Client sends an idempotency key (UUID) in the header. Server stores key + result. On duplicate request with same key: return stored result, don't re-execute. Key expires after 24h.*

04 How do you store and look up idempotency keys efficiently?

■ *Redis with TTL for fast lookup and automatic expiry. Key: idempotency_key. Value: request hash + response. Lock during processing to prevent parallel duplicate execution.*

05 What happens if the server processes the request but crashes before responding?

■ *Client retries with same idempotency key. Server finds the key in storage (if it wrote before crashing) or re-processes (if it didn't). This is why you write the key before processing, not after.*

06 How do you handle idempotency in a message queue consumer?

■ *Messages can be delivered more than once. Consumer should check if the message ID was already processed (store processed IDs). If yes, ack and skip. If no, process then store.*

07 What is the difference between idempotency and exactly-once delivery?

■ *Idempotency is a property of the operation (safe to repeat). Exactly-once delivery is a guarantee from the message system (delivered once). Idempotency is cheaper and more practical.*

08 Can a non-idempotent operation be made safe to retry? How?

■ *Yes: wrap it in a check-then-act pattern with an idempotency key. Check if already executed. If not, execute and store the result. If yes, return the stored result.*

TOPIC 06 ●●●●○ VERY COMMON

Message Queues

Message queue questions test if you understand async architecture, not just whether you've used a tool. Interviewers want to know when you'd use a queue and what breaks without one.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What is the difference between a message queue and pub/sub?

■ *Queue: one producer, one consumer, message deleted after consumption. Pub/sub: one producer, many subscribers, each gets a copy. Kafka blurs the line by retaining messages.*

02 Explain at-most-once, at-least-once, and exactly-once delivery.

■ *At-most-once: fire and forget, may lose messages. At-least-once: retry until acked, may duplicate. Exactly-once: hardest, requires deduplication on both sides. Most systems use at-least-once + idempotent consumers.*

03 What is a dead letter queue and when does a message end up there?

■ *A separate queue for messages that failed after max retries. Message ends there when: processing consistently throws, message is malformed, TTL exceeded. Prevents poison pill messages blocking the queue.*

04 What is backpressure and how do message queues help?

■ *Consumer can't keep up with producer. Without a queue: producer overwhelms consumer, system crashes. With a queue: messages buffer, consumer processes at its own pace, producer isn't blocked.*

05 How do you handle duplicate messages in a consumer?

■ *Every message must have a unique ID. Consumer checks if ID was already processed (DB or Redis). If yes, ack and skip. If no, process then store the ID. This is idempotent consumption.*

06 What is a consumer group in Kafka and how does it enable scaling?

■ *Multiple consumers in a group share partitions of a topic. Each partition consumed by exactly one consumer in the group. Add consumers to scale horizontally. Max parallelism = number of partitions.*

07 When would you use a message queue instead of a direct API call?

■ *When the operation is slow (email, PDF generation), when you need retry logic, when producer and consumer should scale independently, when you need guaranteed delivery.*

08 What is the difference between Kafka and RabbitMQ?

■ *Kafka: log-based, messages retained, consumers control their offset, high throughput, ordered per partition. RabbitMQ: queue-based, messages deleted after ack, push-based, flexible routing.*

TOPIC 07 ●●●●○ VERY COMMON

Distributed Systems Basics

Distributed systems questions separate developers who've built on one server from engineers who've thought about what happens when you have many. Every FAANG and product company interview includes at least two of these.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What is the CAP theorem? Give a real example of a CP and an AP system.

- *Can only guarantee 2 of 3: Consistency, Availability, Partition tolerance. CP: HBase, Zookeeper (refuse requests during partition). AP: DynamoDB, Cassandra (serve stale data during partition).*

02 What is eventual consistency? Give a real-world example of when it's acceptable.

- *All nodes will converge to the same value eventually, not immediately. Acceptable: social media likes, product view counts. Not acceptable: bank balance, inventory during purchase.*

03 What is consistent hashing and why is it used?

- *Maps keys to nodes in a ring. Adding or removing a node only remaps a fraction of keys (1/N) instead of all keys. Used in distributed caches and load balancers to minimise redistribution.*

04 What is a split-brain problem?

- *Network partition splits nodes into two groups, each thinks it's the leader. Both accept writes. When partition heals: conflicting state. Solution: quorum (majority must agree before accepting writes).*

05 What is a distributed lock and what are its failure modes?

- *A lock across multiple service instances (Redis SETNX, Redlock). Failure modes: lock holder dies and holds lock forever (use TTL), clock drift between nodes (Redlock uses multiple Redis nodes).*

06 What is the two-generals problem and why does it matter?

- *Two armies can't reliably agree to attack over an unreliable channel. Proof that guaranteed exactly-once delivery over unreliable networks is impossible. Explains why exactly-once is so hard.*

07 What is a heartbeat in distributed systems?

- *Regular message a node sends to prove it's alive. If heartbeat is missed for N seconds, the node is considered dead and traffic is rerouted. Foundation of failure detection.*

08 What is the difference between a service mesh and a load balancer?

- *Load balancer: distributes traffic, layer 4 or 7, sits in front of services. Service mesh: handles service-to-service communication (retries, circuit breaking, tracing) via sidecar proxies. Complements, doesn't replace.*

TOPIC 08 ●●●●●

ASKED EVERYWHERE

API Design

API design is asked in almost every backend interview because it reveals how you think about contracts, versioning, and the developers who will consume what you build. A good API is harder to design than to build.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What makes an API RESTful? What are the constraints?

■ *Stateless, client-server, cacheable, uniform interface (resource-based URLs, standard methods), layered system, optional code-on-demand. Stateless is the most important — server holds no client session state.*

02 When would you use REST vs GraphQL vs gRPC?

■ *REST: standard web APIs, public APIs. GraphQL: flexible querying, mobile clients with bandwidth concerns, many entity relationships. gRPC: internal service-to-service, high throughput, bi-directional streaming.*

03 How do you version an API without breaking existing clients?

■ *URL versioning (/v1/, /v2/) is most explicit. Header versioning (Accept: application/vnd.api+json;version=2) is RESTful but less discoverable. Never remove/rename fields in a version, only add.*

04 What is cursor-based pagination and why is it better than offset?

■ *Offset: skip N rows — gets slower as N grows, rows shift if data changes during paging. Cursor: encode the last seen row's sort key, fetch rows after it. O(log N), stable even as data changes.*

05 What HTTP status codes should your API return for these scenarios?

■ *200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Server Error. Know all of these.*

06 How do you design an API endpoint that might take 30 seconds to complete?

■ *Async pattern: POST returns 202 Accepted with a job ID immediately. Client polls GET /jobs/:id for status. Or use webhooks: client registers a callback URL, server POSTs result when done.*

07 What is HATEOAS and is it worth implementing?

■ *Hypermedia As The Engine Of Application State: responses include links to next valid actions. Technically RESTful. In practice: rarely implemented, high overhead, most teams skip it consciously.*

08 How do you design backward-compatible API changes?

■ *Add fields, never remove. Make new fields optional. Never change the type of an existing field. Deprecate with a header before removing. Provide migration guides with a sunset date.*

TOPIC 09 ●●●●○

VERY COMMON

Database Design

Database design questions reveal whether you've designed a schema that worked in production or just one that worked in development. The N+1 problem, migrations, and sharding are the three most common deep dives.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 What is database normalisation? What are 1NF, 2NF, 3NF?

■ *1NF: atomic values, no repeating groups. 2NF: 1NF + non-key columns depend on whole primary key. 3NF: 2NF + non-key columns depend only on primary key, not on each other.*

02 When would you denormalise a database and what are the trade-offs?

■ *When read performance is critical and you can tolerate data duplication. Trade-off: faster reads, slower writes, risk of inconsistency. Use carefully, only for measured bottlenecks.*

03 What is the N+1 query problem? How do you detect and fix it?

■ *Fetching N users then making 1 query per user for their posts = N+1 queries. Fix: JOIN or eager loading (ORM's include/with). Detect: log queries, look for repeated queries with different IDs.*

04 When would you choose SQL over NoSQL and vice versa?

■ *SQL: complex relationships, joins, ACID needed, schema is stable. NoSQL: massive scale, flexible schema, simple access patterns, document/graph/time-series data. Not about size — about access patterns.*

05 What is database sharding? What problems does it introduce?

■ *Splitting data across multiple DB instances by a shard key. Problems: cross-shard queries are hard, transactions across shards need 2PC, hot shards if key has poor distribution, resharding is painful.*

06 How do you run a database migration with zero downtime?

■ *Expand-contract pattern: 1) Add new column (nullable). 2) Dual-write to old + new. 3) Backfill old rows. 4) Read from new column. 5) Remove old column. Never: ALTER TABLE that locks in prod.*

07 What is connection pooling and what happens without it?

■ *Reuses existing DB connections instead of creating a new one per request. Without it: TCP handshake + auth overhead per request, DB hits max_connections under load, requests start failing.*

08 What is a foreign key constraint and when would you skip it?

■ *Enforces referential integrity at the DB level. Skip it when: high write throughput (constraint check per insert), sharded databases (cross-shard FK impossible), eventual consistency is acceptable.*

TOPIC 10 ●●●●●

ASKED EVERYWHERE

Production Debugging

This is the most practical category — interviewers ask scenario questions to see if you've actually been paged at 2am and solved a real problem. Structure your answers: observe, hypothesise, test, fix, prevent.

INTERVIEW QUESTIONS + WHAT THEY'RE TESTING

01 Your API response times doubled overnight. Walk me through debugging this.

■ *Check: recent deployments, DB slow query log, CPU/memory/connections, external API calls, cache hit rate. Narrow to: new code path, missing index, connection pool exhaustion, dependency slowdown.*

02 Your database CPU is at 100%. What are the first 5 things you check?

■ *1) pg_stat_activity / SHOW PROCESSLIST for running queries. 2) Slow query log. 3) EXPLAIN on top queries. 4) Missing indexes on recent queries. 5) Recent schema changes. Then: lock contention, autovacuum.*

03 How do you find a memory leak in a production Node.js service?

■ *Take heap snapshots at intervals, compare growing object counts. Tools: clinic.js, --inspect flag + Chrome DevTools. Common causes: event listeners not removed, closures holding references, caches without eviction.*

04 What is distributed tracing and how does it help debug microservices?

■ *Attaches a trace ID to a request that propagates through every service. Each service adds a span. You see the full request journey: which service was slow, what DB queries ran, where the error originated.*

05 What is the difference between a metric, a log, and a trace?

■ *Metric: numeric measurement over time (latency p99, error rate, CPU %). Log: discrete event with context (request ID, user, error message). Trace: end-to-end journey of one request across services.*

06 Your error rate is 1% but you can't reproduce the error. How do you debug it?

■ *Check error logs filtered by the error type. Find a request ID that failed. Trace it: what user, what input, what code path, what was different. Add more logging to the suspected path. Shadow traffic if needed.*

07 How do you debug a race condition that only happens in production?

■ *Add structured logs at both write points with timestamps and request IDs. Analyse logs for overlapping windows. Add a distributed lock or DB-level locking. Load test locally to reproduce at higher concurrency.*

08 What should a good post-mortem contain?

■ *Timeline (what happened, when). Impact (users affected, revenue, duration). Root cause (not 'human error' — the systemic gap). Contributing factors. Action items with owners and dates. Blameless tone throughout.*

MOST CANDIDATES KNOW THE ANSWERS. FEW KNOW THE REASONING.

An interviewer who asks 'what is a deadlock?' isn't looking for a definition. They're looking for whether you've seen one in production, whether you can explain why it happened, and whether you know how to prevent the next one. The definition is the floor. The reasoning is what gets you hired.

HOW TO STUDY THIS

— **Don't memorise answers —** understand the concept until you can explain it to someone who's never heard of it. Then the answer comes naturally.

— **Answer the hint, not just the question —** for each question, read the 'what they're testing' note first. That tells you what the interviewer actually wants to hear.

— **Use the STAR + trade-off format —** for every answer: what it is, why it exists, what problem it solves, and what it trades off. Four sentences, every time.

— **Prioritise the 5-dot topics —** Database Indexing, Caching, API Design, Transactions, and Production Debugging appear in almost every backend interview. Master those five first.

— **Simulate production scenarios —** for debugging questions, don't just answer. Tell a story: 'I'd first check X because Y, then if that's clear I'd look at Z because...' Structure shows seniority.

FINAL THOUGHT

The engineers who do well in backend interviews aren't the ones who've studied the most. They're the ones who've built things, broken things, fixed things, and can talk about the experience honestly. Study this PDF. Then go build something that uses every concept in it.